

DART XML 검색 청크 구현 및 전체 적재 완료

항목	내용
문서 버전	v1.0
작성일	2026-08-26
문서 상태	전체 적재 및 실패 문서 재검증 완료
적용 대상	대회 제공 DART_XML 문서 3,147개
선행 문서	05_dart_xml_parsing_and_validation.md, 06_dart_xml_parsing_persistence.md, 07_disclosure_chunking_rules.md
입력 테이블	disclosure_documents, disclosure_sections, disclosure_content_blocks
출력 테이블	disclosure_chunks, disclosure_chunk_sources
파서 버전	1.1.0
청크 생성기 버전	dart-xml-chunk-v3

1. 문서 목적

이 문서는 파싱된 DART XML을 검색 청크로 변환해 PostgreSQL에 저장한 전체 구현 과정과 최종 검증 결과를 기록한다.

07_disclosure_chunking_rules.md가 청크 생성 원칙과 설계를 정의한다면, 이 문서는 다음 내용을 중심으로 다룬다.

- 실제 구현된 클래스와 연결 흐름
- TEXT·TABLE 청크 생성 방식
- 중첩 표 문맥 보존을 위해 변경한 사항
- 청크와 원문 출처의 DB 저장 방식
- 소규모·전체 배치 수행 과정
- 전체 적재 중 발생한 실패 1건과 해결 방법
- 최종 DB 적재 결과와 다음 작업

2. 검색 청크의 역할

검색 청크는 긴 공시 원문 전체를 질문 처리 단계에 전달하지 않고, 질문과 관련성이 높은 원문 구간을 찾기 위한 검색 단위다.

사용자 질문

→ 기업·기간·공시 후보 검색

→ 관련 검색 청크 조회

- Fact 후보 추출·검증
- 백엔드 계산
- 근거가 연결된 답변 생성

검색 청크는 원문이나 Fact를 대체하지 않는다.

```
disclosure_content_blocks
├─ 파싱 원문과 구조화 TABLE JSONB 보존
├─ disclosure_chunks
│   └─ 질문과 관련된 원문 구간을 찾기 위한 검색 텍스트
└─ 향후 Fact
    └─ 금액·날짜·비율처럼 계산 가능한 정규화 값
```

수치 계산에는 검증된 Fact를 사용하며, 검색 청크는 관련 원문과 Fact 후보 범위를 찾는 데 사용한다.

3. 전체 구현 흐름

```
DisclosureChunkingBatchRunner
→ DisclosureChunkingBatchService
→ DisclosureDocumentChunkingService
→ DisclosureChunkGenerator
    ├─ SectionPathResolver
    ├─ TextChunkGenerator
    │   └─ ChunkTextNormalizer
    │       └─ SentenceBoundarySplitter
    └─ TableChunkGenerator
        ├─ DisclosureTablePayloadReader
        ├─ TableLogicalGridBuilder
        ├─ TableTextSerializer
        ├─ NestedTableContextSelector
        └─ SentenceBoundarySplitter
→ GeneratedDisclosureChunkEntityManager
→ DisclosureChunkPersistenceService
→ disclosure_chunks
→ disclosure_chunk_sources
```

3.1 배치 실행 계층

클래스	역할
<code>DisclosureChunkingBatchRunner</code>	설정값을 읽어 여러 배치를 연속 실행하고 누적 결과를 로그로 출력한다.
<code>DisclosureChunkingBatchService</code>	청킹 대상 문서를 배치 크기만큼 조회하고 문서별 성공·실패 결과를 집계한다.

클래스	역할
<code>DisclosureChunkingBatchRow</code>	문서 한 건의 청킹 결과를 표현한다.
<code>DisclosureChunkingBatchResult</code>	한 배치의 전체·성공·실패·저장 개수를 표현한다.
<code>DisclosureChunkingBatchStatus</code>	문서별 배치 처리 상태를 표현한다.
<code>TargetDisclosureChunkingRunner</code>	특정 문서 UUID 한 건만 재청킹할 때 사용한다.

3.2 문서 단위 처리 계층

`DisclosureDocumentChunkingService`는 문서 한 건의 청킹 유스케이스를 담당한다.

문서 조회
 → 파싱 완료 여부 확인
 → `Section`과 `ContentBlock` 조회
 → 청크 생성
 → 기존 청크를 새 결과로 교체 저장
 → 성공 시 `COMPLETED`
 → 실패 시 `FAILED`와 최하위 예외 메시지 기록

`DisclosureChunkFailureRecorder`는 본 처리 트랜잭션과 분리된 트랜잭션에서 실패 상태를 기록한다. 따라서 청크 저장 트랜잭션이 롤백되더라도 실패 원인은 DB에 남는다.

3.3 청크 생성 계층

`DisclosureChunkGenerator`는 문서 전체 블록을 원문 순서로 정렬하고 최종 `chunkSequenceNo`를 1부터 연속 부여한다.

- `HEADING·PARAGRAPH`는 `TextChunkGenerator`로 전달한다.
- `TABLE`은 `TableChunkGenerator`로 전달한다.
- `TABLE`을 만나면 누적 중인 `TEXT` 청크를 먼저 종료한다.
- `Section` 경계를 넘어서 `TEXT`를 결합하지 않는다.
- `PAGE_BREAK`는 독립 청크를 만들지 않는다.
- 현재 `IMAGE` 블록은 독립 검색 청크를 만들지 않는다.

4. TEXT 청크 구현

`TextChunkGenerator`는 같은 `Section`과 `HEADING` 문맥 안의 `PARAGRAPH`를 결합하거나 분할한다.

4.1 처리 규칙

- `HEADING`은 단독 본문이 아니라 뒤따르는 문단의 검색 문맥으로 사용한다.
- 짧은 인접 문단은 목표 길이까지 결합한다.
- 긴 단일 문단은 `SentenceBoundarySplitter`가 문장·줄바꿈·구두점 경계를 우선해 나눈다.
- 마지막 수단에서만 절대 글자 수 경계를 사용한다.
- 원본 `ContentBlock`과 원문 행 범위를 출처로 보존한다.

4.2 길이 정책

설정	값
목표 하한	700자
목표 상한	1,000자
일반 최대	1,400자
절대 최대	2,000자

최종 적재 결과에서 가장 긴 TEXT `body_text`는 1,987자로 절대 최대 길이를 만족했다.

5. TABLE 청크 구현

TABLE은 `disclosure_content_blocks.structured_content`의 JSONB를 읽어 검색 가능한 행 텍스트로 변환한다.

5.1 처리 단계

```
TABLE JSONB
→ DisclosureTablePayloadReader
→ ParsedDisclosureTable
→ TableLogicalGridBuilder
→ LogicalTableGrid
→ TableTextSerializer
→ SerializedTable
→ TableChunkGenerator
→ TABLE 청크 후보
```

5.2 논리 그리드

`TableLogicalGridBuilder`는 `rowSpan`과 `colSpan`을 펼쳐 각 행에서 상위 병합 셀의 문맥을 확인할 수 있는 논리 그리드를 만든다.

원본 JSONB는 변경하지 않고 검색용 표현에서만 셀 문맥을 보완한다.

5.3 직렬화와 분할

- 셀은 논리적 열 순서대로 | 구분자를 사용해 직렬화한다.
- 가능한 한 행 단위로 묶고 나눈다.
- 연속된 선두 HEADER 전용 행은 분할된 각 청크에 반복한다.
- 긴 단일 행은 문장 경계 기준으로 여러 조각으로 나눈다.
- 각 청크는 원본 TABLE 블록, 중첩 경로와 원본 행 범위를 보존한다.

5.4 길이 정책

설정	값
목표 하한	1,000자
목표 상한	1,500자

설정	값
일반 최대	2,000자
절대 최대	3,000자

최종 적재 결과에서 가장 긴 TABLE `body_text`는 2,978자로 절대 최대 길이를 만족했다.

6. 중첩 표 문맥 보존 개선

6.1 처음 구현의 문제

초기 구현은 중첩 표의 상위 셀 전체 텍스트를 검색 문맥에 넣었다. 부모 셀이 매우 길면 다음 문제가 발생했다.

- `search_text`가 지나치게 길어졌다.
- 중첩 표와 직접 관련 없는 먼 문장이 함께 들어갔다.
- 단순 글자 수 절단 시 가까운 핵심 문맥이 사라질 수 있었다.

6.2 파서 스키마 v2

`ParsedDisclosureTableContext`를 추가하고 중첩 표의 바로 앞·뒤 텍스트를 파서에서 보존했다.

```
ParsedDisclosureTable
├─ parentContext
│  └─ precedingText
│  └─ followingText
```

TABLE JSONB는 `schemaVersion=2`를 사용한다. `DisclosureTablePayloadReader`는 기존 v1과 신규 v2를 모두 읽을 수 있게 유지했다.

6.3 검색 문맥 선택

`NestedTableContextSelector`는 다음 순서로 중첩 표의 상위 문맥을 선택한다.

1. 중첩 표 바로 앞 문장을 우선한다.
2. 필요하면 바로 뒤 문장을 추가한다.
3. 여러 단계의 중첩에서는 현재 표와 가장 가까운 부모 문맥을 우선한다.
4. 누적 상위 셀 문맥은 최대 500자로 제한한다.
5. 잘린 문맥에는 말줄임표를 사용한다.

이 변경을 적용하기 위해 v1 중첩 표가 존재했던 DART XML 문서 661개를 선별 재파싱했다.

7. 저장 모델과 트랜잭션

7.1 `disclosure_chunks`

문서별 최종 검색 청크를 저장한다.

주요 정보:

- 소속 문서와 Section

- TEXT, TABLE 청크 유형
- 문서 내 연속 청크 순번
- Section 전체 경로
- 검색 본문 `body_text`
- 문맥이 추가된 `search_text`
- 본문·검색 문자열 길이
- 생성기 이름과 버전

7.2 disclosure_chunk_sources

청크가 어느 파싱 원문에서 만들어졌는지 저장한다.

주요 정보:

- 소속 청크
- 원본 ContentBlock
- 청크 안의 출처 순서
- 원본 Block 순번
- 원문 시작·종료 행
- 중첩 표 JSON 경로
- TABLE 원본 행 시작·종료 인덱스

7.3 문서 단위 전체 교체

`DisclosureChunkPersistenceService`는 다음 작업을 하나의 트랜잭션으로 수행한다.

```

기존 문서 청크 삭제
→ 새 청크와 출처 저장
→ DB 제약조건 확인을 위한 flush
→ 문서 chunk_status=COMPLETED
  
```

중간에 실패하면 기존 청크 삭제를 포함해 전체 작업이 롤백된다. 따라서 한 문서의 청크가 일부만 저장된 상태로 남지 않는다.

8. 배치 실행 설정

주요 설정은 다음과 같다.

환경변수	역할
<code>FOLIOLENS_CHUNKING_ON_STARTUP</code>	시작 시 청킹 배치 실행 여부
<code>FOLIOLENS_CHUNKING_BATCH_SIZE</code>	한 번에 조회·처리할 문서 수
<code>FOLIOLENS_CHUNKING_MAX_DOCUMENTS</code>	한 실행에서 처리할 최대 문서 수
<code>FOLIOLENS_CHUNKING_MAX_FAILURES</code>	중단 전 허용할 최대 실패 수

전체 적재가 끝난 일반 실행 환경에서는 `FOLIOLENS_CHUNKING_ON_STARTUP=false`로 둔다.

특정 문서 재처리는 `TargetDisclosureChunkingRunner`를 사용한다. 문서 UUID를 직접 지정하므로 전체 배치를 다시 실행할 필요가 없다.

9. 전체 적재 중 발생한 실패와 해결

9.1 실패 문서

항목	값
기업	POSCO홀딩스
공시	[기재정정]사업보고서 (2023.12)
접수번호	20240322000986
문서 UUID	90251256-41af-4a16-8b21-ff3cadf7a7db
실패 당시 파싱 상태	COMPLETED
실패 원인	반복 TABLE 머리글 길이 7,225자가 절대 최대 3,000자를 초과

당시 예외:

```
IllegalStateException: TABLE 머리글만으로 절대 최대 길이를 초과했습니다.  
headerLength=7225
```

실패 문서에는 청크가 0개 저장돼 있었으므로 부분 저장이나 데이터 오염은 없었다.

9.2 해결 정책

절대 최대 길이를 단순히 7,225자 이상으로 늘리지 않았다. 긴 머리글을 모든 청크에 반복하면 저장량이 증가하고 검색 청크가 지나치게 커지기 때문이다.

다음 공통 폴백을 `TableChunkGenerator`에 추가했다.

```
반복 머리글이 절대 최대 길이를 소진함  
→ 머리글 반복 해제  
→ 머리글을 버리지 않고 전체 행 목록에 포함  
→ 모든 행을 원래 순서대로 일반 TABLE 행처럼 분할  
→ 긴 단일 머리글 행은 기존 문장 경계 분할 사용  
→ 원본 행과 출처 보존
```

이 조건에 해당하지 않는 정상 표의 처리 경로는 변경하지 않았다.

9.3 실제 문서 재검증

수정된 이미지로 실패 문서 한 건을 실제 재청킹했다.

항목	결과
저장 청크	22,980개
저장 출처	44,947개
최대 <code>body_text</code> 길이	2,978자
최종 상태	COMPLETED
오류 메시지	NULL

10. 최종 DB 결과

2026-08-26 최종 확인 결과다.

10.1 문서 처리 상태

상태	문서 수
COMPLETED	3,147
FAILED	0
합계	3,147

모든 완료 문서의 `chunk_generator_version`은 `dart-xml-chunk-v3`다.

10.2 청크 유형별 결과

유형	청크 수	최소 본문 길이	최대 본문 길이
TEXT	406,831	1자	1,987자
TABLE	985,590	2자	2,978자
합계	1,392,421	-	-

10.3 출처와 무결성

검증 항목	결과
전체 청크 출처	2,194,382개
빈 <code>body_text</code>	0개
유형별 절대 최대 길이 초과	0개
청킹 실패 문서	0개

11. 재실행 결과의 동일성

같은 원문, 같은 파싱 결과와 같은 `dart-xml-chunk-v3` 설정으로 초기 DB에서 다시 실행하면 다음 값은 결정적으로 동일하게 생성된다.

- 문서별 청크 개수와 순서
- `chunk_type`
- `body_text`
- `search_text`
- Section 경로
- 원문 Block과 행 범위
- 청크와 출처의 연결 구조

다음 저장 기술값은 재실행 시 달라질 수 있다.

- 청크와 출처의 UUID
- DB를 처음부터 재적재하면 문서·Section·Block UUID
- `created_at`, `updated_at`, `chunked_at`

즉 검색과 근거 추적에 사용되는 의미 데이터는 동일하지만, DB 내부 식별자와 생성 시각까지 동일한 것은 아니다.

12. 테스트 및 검증

다음 범위를 검증했다.

- Section 경로 생성
- 텍스트 정규화
- 문장 경계 분할
- TEXT 결합·분할
- rowspan-colspan 논리 그리드
- TABLE 직렬화
- TABLE 행 단위 분할과 머리글 반복
- 긴 단일 TABLE 행 분할
- 절대 최대 길이를 소진한 머리글 폴백
- 중첩 표 재귀 처리
- 중첩 표 앞·뒤 문맥 선택
- 생성 결과 정렬과 연속 순번
- 파싱 모델에서 엔티티로의 매핑
- 문서 단위 전체 교체 저장과 롤백
- 실패 상태 별도 기록
- 소규모 및 전체 배치 처리

실행 결과:

```
TableChunkGeneratorTest: BUILD SUCCESSFUL
청킹 관련 전체 테스트(*Chunk*): BUILD SUCCESSFUL
실패 문서 실제 재청킹: 성공
최종 DB 상태: COMPLETED 3,147 / FAILED 0
```

13. 현재 완료 범위와 남은 범위

13.1 완료

- DART XML 전체 구조 조사
- DART XML 전체 파싱 검증
- Section·ContentBlock·TABLE JSONB 저장
- 중첩 표 앞·뒤 문맥 보존
- TEXT·TABLE 검색 청크 생성
- 청크와 원문 출처 연결
- 문서 단위 트랜잭션 저장
- 소규모·전체 배치 실행
- 전체 DART XML 3,147개 청킹 완료

13.2 후속 작업

1. 기업·기간·공시 메타데이터를 이용한 후보 문서 검색
2. `disclosure_chunks.search_text` 기반 청크 검색
3. 검색 결과 순위화와 앞뒤 문맥 확장
4. 대표 질문을 이용한 top K 검색 품질 평가
5. 대표 공시 유형의 Fact 스키마와 추출기 구현
6. 결정적 계산 로직 연결
7. 검색·Fact·계산 기능을 LLM 호출 도구로 제공

현재 독립 IMAGE 블록의 `IMAGE_CAPTION` 청크, HTML·PDF 청킹과 검색 인덱스는 구현 범위에 포함되지 않았다. 필요성과 데이터 가치를 검증한 뒤 후속 버전에서 구현한다.

14. 결론

DART XML 원문은 다음 단계까지 모두 완료됐다.

```
원문 경로 등록
→ XML 구조 조사
→ XML 파싱
→ Section·ContentBlock·TABLE JSONB 저장
→ 중첩 표 문맥 보존
→ TEXT·TABLE 검색 청크 생성
→ 원문 출처 연결
→ PostgreSQL 전체 적재
→ 실패 문서 보완 및 재검증
```

현재 DB에는 DART XML 3,147개 문서의 검색 청크와 원문 출처가 모두 저장돼 있다. 다음 개발 단계는 이 청크를 질문 조건에 따라 실제로 찾아내는 백엔드 검색 기능이다.